Contest Log Analytics - Installation Guide

Version: 1.0.0-alpha.18 **Date:** 2026-01-24 **Last Updated:** 2026-01-31 **Category:** User Guide

---

--- Revision History ---

[1.0.0-alpha.18] - 2026-01-31

Changed

- Updated version to match project release v1.0.0-alpha.18 (no content changes)

[1.0.0-alpha.17] - 2026-01-30

Changed

- Updated version to match project release v1.0.0-alpha.17 (no content changes)

[1.0.0-alpha.16] - 2026-01-30

Changed

- Updated version to match project release v1.0.0-alpha.16 (no content changes)

[1.0.0-alpha.15] - 2026-01-30

Changed

- Updated version to match project release v1.0.0-alpha.15 (no content changes)

[1.0.0-alpha.14] - 2026-01-26

Changed

- Updated version to match project release v1.0.0-alpha.14 (no content changes)

[1.0.0-alpha.13] - 2026-01-26

Changed

- Updated version to match project release v1.0.0-alpha.13 (no content changes)

[1.0.0-alpha.11] - 2026-01-24

Changed

- Updated version to match project release v1.0.0-alpha.11 (no content changes)

[1.0.0-alpha.10] - 2026-01-24

Changed

- Updated version to match project release v1.0.0-alpha.10 (no content changes)

[1.0.0-alpha.1] - 2026-01-13

Changed

- Updated version to match project release versioning.
- Enhanced Docker installation section with detailed information about:
  - Data files location and requirements
  - Directory structure expectations
  - Automatic environment variable configuration
  - Note about included sample data files

[0.126.0-Beta] - 2025-12-18

Changed

- Promoted the Docker installation workflow to the primary "Installation Steps".

- Demoted the existing Conda/CLI instructions to "Advanced / Developer Setup".

[0.94.1-Beta] - 2025-12-07

Changed

- Added `python-kaleido` to the required libraries in Step 3 to support

static image export for Plotly reports.

[0.94.0-Beta] - 2025-12-06

Changed

- Added `plotly` to the required libraries in Step 3 to support the

Phase 2 visualization engine upgrade.

[0.91.15-Beta] - 2025-10-11

Fixed

- Corrected data file dependencies to state that NAQPmults.dat is

not required for the CQ 160-Meter contest.

[0.91.14-Beta] - 2025-10-10

Fixed

- Added missing `CQ160mults.dat` to the list of required data files.

[0.90.13-Beta] - 2025-10-06

Fixed

- Added missing `requests` and `beautifulsoup4` libraries to the conda

installation command in Step 3. These are required by the cty_manager

For Web Dashboard (Preferred)

- **Docker Desktop**: The application runs inside a container, requiring no local Python or library installation.

---

2. Installation Steps

Method A: Web Dashboard (Docker)

This is the easiest way to get started. Docker automatically handles all dependencies and environment configuration.

1. **Clone the Repository:**

   ```
   git clone https://github.com/user/Contest-Log-Analyzer.git
   cd Contest-Log-Analyzer
   ```

2. **Verify Data Files:** The repository includes sample data files in `CONTEST_LOGS_REPORTS/data/`. These are sufficient for initial testing. The required data files include:

   - `cty.dat`: Required for all contests.
   - `CQ160mults.dat`: Required for the CQ 160-Meter contest.
   - `arrl_10_mults.dat`: Required for the ARRL 10 Meter contest.
   - `ARRLDXmults.dat`: Required for the ARRL DX contest.
   - `NAQPmults.dat`: Required for NAQP contests.
   - `SweepstakesSections.dat`: Required for ARRL Sweepstakes and ARRL Field Day.
   - `band_allocations.dat`: Required for all contests to perform frequency validation.
   - `iaru_officials.dat`: Required for the IARU HF World Championship contest.

   Note: If you need to add your own data files, place them in `CONTEST_LOGS_REPORTS/data/` before starting Docker.

3. **Start the Application:**

   ```
   docker-compose up --build
   ```

   Docker automatically configures the environment variables:

   - `CONTEST_INPUT_DIR=/app/CONTEST_LOGS_REPORTS`
   - `CONTEST_REPORTS_DIR=/app/CONTEST_LOGS_REPORTS`

   The application expects:

   - Data files in: `CONTEST_LOGS_REPORTS/data/`
   - Log files in: `CONTEST_LOGS_REPORTS/Logs/`
   - Reports generated in: `CONTEST_LOGS_REPORTS/reports/`

4. **Access the Dashboard:**
    - Development: Open your web browser and navigate to `http://localhost:8000`.
    - Production: Visit `https://cla.kd4d.org`.

---

3. Managing and Updating Docker Containers

This section covers managing Docker containers for remote hosting and production deployment, including updating to new releases and handling common issues.

3.1. Updating to a New Release (Remote Hosting)

Simplified Update Process:

For most updates, you only need these three commands (replace `v1.0.0-alpha.XX` with the new version tag):

```
cd /docker/Contest-Log-Analyzer
git fetch --tags origin
git checkout v1.0.0-alpha.XX
docker compose up --build -d
```

Note: Adjust the path (`/docker/Contest-Log-Analyzer`) to match your installation directory. If you're using a custom compose file name, add `-f docker-compose.prod.yml` (or your filename) to the docker compose command.

What this does:

- Fetches the latest tags from the repository
- Checks out the specific release version
- Rebuilds the Docker image with the new code
- Stops old containers and starts new ones automatically (no need to manually stop)

After updating: Check if your site works. The commands below are only needed if something breaks (see Section 3.2).

3.2. Troubleshooting After Update

When to use these commands: Only run these if something breaks after updating. Most updates work fine with just the three commands in Section 3.1.

If Static Files (CSS/JS) Aren't Loading If your site looks broken (missing styles, JavaScript not working), collect static files:

```
docker compose exec web python web_app/manage.py collectstatic --noinput
```

(Add `-f docker-compose.prod.yml` if using a custom compose file)

If You See Database Migration Warnings If you see errors like:

```
You have unapplied migration(s). Your project may not work properly
until you apply the migrations.
```

Run migrations:

```
docker compose exec web python web_app/manage.py migrate
```

(Add `-f docker-compose.prod.yml` if using a custom compose file)

If Containers Won't Start

1. **Check logs for errors:**

   ```
   docker compose logs web
   ```

2. **Check if port is already in use:**

   ```
   sudo netstat -tulpn | grep 8000
   ```

3. **Force recreate containers:**

   ```
   docker compose up --build --force-recreate -d
   ```

4. **View logs to diagnose issues:**

   ```
   docker compose logs -f web
   ```

   See Section 3.3 below for detailed log viewing options.

Understanding Docker Exec Commands Why `docker compose exec`? These commands run **inside the container** where Python and Django are installed. You cannot run Django management commands from your host system unless Python is installed there (which is not required for Docker deployment).

Alternative: Interactive Shell (if you need to run multiple commands):

```
docker compose exec web bash
```

Then inside the container:

```
cd /app
python web_app/manage.py migrate
python web_app/manage.py collectstatic --noinput
exit
```

3.3. Troubleshooting Container Issues

View container logs:

After running `docker compose up --build -d`, you can view container logs with:

Follow logs in real-time (recommended):

```
docker compose logs -f web
```

The `-f` flag follows logs in real time (like `tail -f`). Press `Ctrl+C` to exit.

View logs once:

```
docker compose logs web
```

Shows logs and exits immediately.

View last N lines:

```
docker compose logs --tail 50 web
```

Shows the last 50 lines of logs.

View all services:

```
docker compose logs -f
```

Shows logs from all services in the compose file.

If using a custom compose file:

```
docker compose -f docker-compose.prod.yml logs -f web
```

Check container status:

```
docker compose ps
```

Shows which containers are running and their status.

Check if containers are running:

```
docker ps
```

Shows running containers with their names, ports, and status.

Check container resource usage:

```
docker stats
```

Shows real-time CPU, memory, and network usage for all running containers.

Restart containers without rebuilding:

```
docker-compose restart
```

Restarts all services without rebuilding images. Use this when only configuration changes are needed.

Restart with rebuild (after code changes):

```
docker-compose down
docker-compose up --build -d
```

Stops containers, rebuilds images, and restarts in detached mode. Use this after updating code or dependencies.

Restart specific container (after settings changes):

```
docker restart contest-log-analyzer-web-1
# or
docker restart web-1
```

(Replace with your actual container name - use `docker ps` to find it)

Use this when you've made configuration changes inside the container (e.g., Django settings) and only need to restart the web container without rebuilding.

Access container shell for debugging:

```
docker exec -it web-1 bash
```

Opens an interactive bash shell inside the container for troubleshooting.

Remove all containers and start fresh:

```
# Stop and remove containers, networks
docker-compose down

# Remove volumes (CAUTION: Deletes data in volumes)
docker-compose down -v

# Rebuild and start fresh
docker-compose up --build -d
```

Fix "413 Request Entity Too Large" error when uploading logs:

If you see this error when uploading log files (especially multiple files):

```
413 Request Entity Too Large
nginx
```

This means nginx is blocking the upload because the combined file size exceeds nginx's default limit (usually 1MB). Contest log files can be several megabytes each, so uploading 3 files easily exceeds this limit.

Solution: Increase nginx `client_max_body_size`

Important: If you're using HTTPS (SSL/TLS), you likely have **two separate server blocks** - one for HTTP (port 80) and one for HTTPS (port 443). You

need to add `client_max_body_size` to **both** blocks. Certbot (Let's Encrypt) typically creates a separate HTTPS server block when setting up SSL.

1. **Locate your nginx configuration file:**

   - Common locations:
     - `/etc/nginx/sites-available/default` (default site)
     - `/etc/nginx/sites-available/cla.kd4d.org` (site-specific config - Certbot often creates this)
     - `/etc/nginx/nginx.conf` (main config, less common for per-site settings)
     - `/etc/nginx/conf.d/default.conf` (alternative location)

   To find your site config:

   ```
   # Search for your domain
   grep -r "cla.kd4d.org" /etc/nginx/sites-available/

   # List available site configs
   ls -la /etc/nginx/sites-available/
   ```

2. **Edit the nginx configuration:**

   ```
   sudo nano /etc/nginx/sites-available/cla.kd4d.org
   # or
   sudo nano /etc/nginx/sites-available/default
   ```

3. **Find both server blocks (HTTP and HTTPS):**

   Look for two `server {` blocks in the file:

   - One with `listen 80;` (HTTP)
   - One with `listen 443 ssl;` (HTTPS - created by Certbot)

   Example structure:

   ```
   server {
       listen 80;
       server_name cla.kd4d.org;
       # ... other settings ...
   }

   server {
       listen 443 ssl;
       server_name cla.kd4d.org;
       # ... SSL certificates, other settings ...
   }
   ```

4. **Add `client_max_body_size` to BOTH server blocks:**

   HTTP server block (port 80):

```
server {
    listen 80;
    server_name cla.kd4d.org;

    client_max_body_size 50M;  # ADD THIS LINE

    # ... other settings ...
}
```

HTTPS server block (port 443):

```
server {
    listen 443 ssl;
    server_name cla.kd4d.org;

    client_max_body_size 50M;  # ADD THIS LINE (required for HTTPS uploads!)

    # ... SSL certificates, other settings ...
}
```

Note: Even though users access your site via HTTPS (port 443), you should add it to both blocks for consistency. The HTTPS block is the critical one if your site is accessed via `https://`.

Alternative: Add to `http` block (applies to all sites): If you want this setting to apply to all sites, you can add it once in the `http {` block at the top of the file or in `/etc/nginx/nginx.conf`:

```
http {
    # ... other directives ...
    client_max_body_size 50M;  # Applies to all sites
    # ... rest of config ...
}
```

5. **Test nginx configuration:**

   ```
   sudo nginx -t
   ```

   This will verify your configuration syntax is correct before reloading.

   If you see errors: Check that:

   - Both server blocks have valid syntax
   - The `location` blocks have paths (e.g., `location / {` not just `location {`)
   - No missing semicolons or brackets

6. **Reload nginx:**

   ```
   sudo systemctl reload nginx
   # or
   sudo service nginx reload
   ```

Important: After editing, you must reload nginx for changes to take effect. If you only edit the file without reloading, the old configuration is still active.

Recommended values:

- `50M` (50 megabytes) - Good for most contest logs
- `100M` (100 megabytes) - For larger contests or multiple large files
- `0` (unlimited) - Not recommended for security reasons

Note: After increasing the limit, you may also need to increase Django's `FILE_UPLOAD_MAX_MEMORY_SIZE` and `DATA_UPLOAD_MAX_MEMORY_SIZE` in `web_app/config/settings.py` if those limits are also being hit. The default Django limits are typically 2.5MB, but Django settings should already be configured for larger uploads in the container.

Fix Django CSRF "Trusted Origin" error for HTTPS domain:

If you see errors like:

`Django protecting you: your site is now being accessed via https://cla.kd4d.org, but Django doesn't trust that origin yet.`

This means Django's CSRF protection doesn't recognize your production domain. You need to add it to `CSRF_TRUSTED_ORIGINS` in Django settings.

Step-by-Step Process:

1. **Identify the container name:**

   ```
   docker ps
   ```

   Look for your web container name (e.g., `contest-log-analyzer-web-1` or `web-1`).

2. **Open a shell inside the container:**

   ```
   docker exec -it contest-log-analyzer-web-1 bash
   # or
   docker exec -it web-1 bash
   ```

   (Replace with your actual container name)

3. **Navigate to the project directory:**

   ```
   cd /app
   ```

4. **Edit Django settings file:**

   ```
   nano web_app/config/settings.py
   ```

5. **Find or add `CSRF_TRUSTED_ORIGINS`:**

   If `CSRF_TRUSTED_ORIGINS` doesn't exist, add it after the `ALLOWED_HOSTS` line:

```
ALLOWED_HOSTS = ['*']

CSRF_TRUSTED_ORIGINS = [
    "https://cla.kd4d.org",
]
```

If `CSRF_TRUSTED_ORIGINS` already exists, add your domain to the list:

```
CSRF_TRUSTED_ORIGINS = [
    "https://cla.kd4d.org",  # Add this line
    # ... other origins if any ...
]
```

Important: Include the full URL with `https://` scheme. Do not include a trailing slash.

6. **Save and exit nano:**
   - Press `Ctrl+O` to save
   - Press `Enter` to confirm
   - Press `Ctrl+X` to exit
7. **Exit the container shell:**

   ```
   exit
   ```
8. **Restart the web container:**

   ```
   docker restart contest-log-analyzer-web-1
   # or
   docker restart web-1
   ```

   (Replace `contest-log-analyzer-web-1` or `web-1` with your actual container name - use `docker ps` to find it)

   Alternatively, you can restart all containers:

   ```
   docker-compose restart
   ```

   Note: After restarting, the Django settings changes will take effect. You may need to wait a few seconds for the container to fully restart before testing.
9. **Verify the fix:** Reload your browser page and try the upload again. The CSRF error should be resolved.

Note about `ALLOWED_HOSTS`:

Your settings currently have `ALLOWED_HOSTS = ['*']`, which works functionally but is not ideal for security. For better security, consider changing it to:

```
ALLOWED_HOSTS = ['cla.kd4d.org', 'localhost', '127.0.0.1']
```

However, `ALLOWED_HOSTS` and `CSRF_TRUSTED_ORIGINS` serve different purposes:

- `ALLOWED_HOSTS`: Controls which hosts Django will accept requests from (security against Host header attacks)
- `CSRF_TRUSTED_ORIGINS`: Controls which origins Django trusts for CSRF validation (required for HTTPS domains)

Both may need to be configured, but `CSRF_TRUSTED_ORIGINS` is the one causing your current error.

Method B: Developer Setup (For Code Development)

Use this method if you are developing the code and need direct access to the Python environment.

Step 1: Clone the Repository Open a terminal or command prompt, navigate to the directory where you want to store the project, and clone the remote Git repository. **CODE_BLOCK** git clone https://github.com/user/Contest-Log-Analyzer.git cd Contest-Log-Analyzer **CODE_BLOCK** This will create the project directory (`Contest-Log-Analyzer`) on your local machine.

Step 2: Create and Activate the Conda Environment It is a best practice to create an isolated environment for the project's dependencies. This prevents conflicts with other Python projects on your system. **CODE_BLOCK**

Create an environment named "cla" with Python 3.11

conda create --name cla python=3.11

Activate the new environment

conda activate cla **CODE_BLOCK**

Step 3: Install Libraries with Conda With the `cla` environment active, use the following single command to install all required libraries from the recommended `conda-forge` channel. This includes `plotly` for interactive charts. **CODE_BLOCK** conda install -c conda-forge pandas numpy matplotlib seaborn imageio prettytable tabulate requests beautifulsoup4 plotly **CODE_BLOCK**

Step 4: Set Up the Input and Output Directories The application requires separate directories for its input files (logs, data) and its output files (reports). This separation is critical to prevent file-locking issues with cloud sync services.

1. Create the Input Directory: This folder will contain your log files and required data files. It can be located anywhere, including inside a cloud-synced folder like OneDrive. Example: `C:\Users\YourUser\OneDrive\Desktop\CLA_Inputs` Inside this folder, you must create the following subdirectories: **CODE_BLOCK** CLA_Inputs/ | +-- data/ | +-- Logs/ **CODE_BLOCK**

2. Create the Output Directory: This folder is where the analyzer will save all generated reports. **This directory must be on a local, non-synced path.** A recommended location is in your user profile directory. Example: `%USERPROFILE%\HamRadio\CLA` (which translates to `C:\Users\YourUser\HamRadio\CLA`)

Step 5: Configure Django Settings For local development, the web application uses Django settings. The default settings in `web_app/config/settings.py` should work for most development scenarios. For production deployment, see the Docker installation method above.

Step 6: Obtain and Place Data Files The analyzer relies on several external data files. Download the following files and place them inside the `data/` subdirectory within your **Input Directory** (`CONTEST_INPUT_DIR`).

- `cty.dat`: Required for all contests.
- `CQ160mults.dat`: Required for the CQ 160-Meter contest.
- `arrl_10_mults.dat`: Required for the ARRL 10 Meter contest.
- `ARRLDXmults.dat`: Required for the ARRL DX contest.
- `NAQPmults.dat`: Required for NAQP contests.
- `SweepstakesSections.dat`: Required for ARRL Sweepstakes and ARRL Field Day.
- `band_allocations.dat`: Required for all contests to perform frequency validation.
- `iaru_officials.dat`: Required for the IARU HF World Championship contest.

---

4. Running the Analyzer

For Docker Installation (Recommended)

1. **Start the application:**

   ```
   docker-compose up --build
   ```

2. **Access the web interface:**

   - Development: `http://localhost:8000`
   - Production: `https://cla.kd4d.org`

For Developer Setup

1. **Activate your conda environment:**

   `conda activate cla`

2. **Run Django migrations (if needed):**

   `python web_app/manage.py migrate`

3. **Start the development server:**

   `python web_app/manage.py runserver`

4. **Access the web interface:**

   - Open your browser to `http://localhost:8000`